

Héberger le site StatDirm

Trois façons d'héberger l'application : **Docker** (recommandé), **build manuel + serveur**, ou **plateforme type Netlify/Vercel**.

Option 1 : Docker (recommandé)

Idéal sur un VPS (OVH, Scaleway, DigitalOcean, etc.) ou une machine avec Docker.

En local (test)

```
# Build + lancement
npm run docker:prod:build

# Le site est sur http://localhost
```

En production sur un serveur

1. Cloner le projet sur le serveur

```
git clone <url-du-repo> StatDirm && cd StatDirm
```

2. Créer un `.env` (identifiants de connexion à l'app)

```
cp .env.example .env
nano .env
```

Exemple :

```
VITE_APP_USERNAME=admin
VITE_APP_PASSWORD=votre-mot-de-passe-securise
```

Les variables `VITE_APP_*` sont prises au **moment du build**. Il faut donc **rebuild l'image** après avoir modifié le `.env`.

3. Builder et lancer avec Docker Compose

```
docker compose -f docker-compose.prod.yml up --build -d
```

L'app écoute sur le **port 80**. Pour un autre port, modifier `ports` dans `docker-compose.prod.yml` (ex. `"8080:80"`).

4. Arrêter

```
docker compose -f docker-compose.prod.yml down
```

Note : Si le build Docker échoue à cause de `package-lock.json`, retirer `package-lock.json` de `.dockerignore` ou générer un `package-lock.json` avec `npm install` puis recommencer le build.

Option 2 : Build manuel + Nginx (ou autre serveur web)

Sur un serveur où tu préfères ne pas utiliser Docker.

1. Build de l'app

En local ou sur le serveur :

```
npm ci
npm run build
```

Le build est dans le dossier `build/` .

2. Déployer les fichiers

Copier **tout le contenu** de `build/` vers le répertoire servi par ton serveur web, par exemple :

- **Nginx :** `/var/www/statdirm/`
- **Apache :** `/var/www/html/statdirm/`

Exemple avec `rsync` :

```
rsync -avz build/ user@ton-serveur:/var/www/statdirm/
```

3. Configurer Nginx

Exemple de bloc pour une app SPA (React) :

```
server {
    listen 80;
```

```
server_name ton-domaine.fr;
root /var/www/statdirm;
index index.html;

location / {
    try_files $uri $uri/ /index.html;
}

location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}
}
```

Puis recharger Nginx : `sudo nginx -t && sudo systemctl reload nginx` .

Identifiants : ils sont figés dans le build via les variables `VITE_APP_USERNAME` et `VITE_APP_PASSWORD` . Pour les changer, modifier le `.env` , refaire `npm run build` et redéployer le contenu de `build/` .

Option 3 : Netlify / Vercel / GitHub Pages

Pour un hébergement statique gratuit (front uniquement).

Netlify

Le projet contient un fichier `netlify.toml` : Netlify utilise automatiquement la commande de build, le dossier de publication et les redirects SPA.

1. Compte sur netlify.com, connexion au dépôt Git (GitHub/GitLab/Bitbucket).
2. **Nouveau site** → **Importer un projet** : choisir le dépôt. Les paramètres sont lus depuis `netlify.toml` :
 - **Build command** : `npm run build`
 - **Publish directory** : `build`
3. **Variables d'environnement** (Site settings → Environment variables) : `VITE_APP_USERNAME` , `VITE_APP_PASSWORD` (pour le build ; optionnel si valeurs par défaut en dev).
4. Déploiement automatique à chaque push sur la branche connectée.

Afficher les données en production : l'app charge les données à l'exécution. Pour afficher les effectifs réels sur Netlify :

- **Option A** : placer le fichier `public/data/agents.json` (même format que pour le build local) dans le dépôt. Il sera déployé et servi ; l'app le charge au chargement de la page.
- **Option B** : héberger le JSON ailleurs et définir la variable d'environnement `VITE_APP_DATA_URL` (URL complète du JSON) au build.

Sans `public/data/agents.json` ni `VITE_APP_DATA_URL`, le tableau de bord s'affiche avec des données vides (repli).

Vercel

1. Compte sur vercel.com, import du projet Git.
2. Framework : **Vite** (détecté automatiquement).
3. **Output Directory** : `build`.
4. Variables d'environnement : idem (`VITE_APP_USERNAME`, `VITE_APP_PASSWORD`).

GitHub Pages

1. Dans le dépôt : **Settings** → **Pages**.
2. Source : **GitHub Actions**.
3. Créer le workflow `.github/workflows/deploy.yml` (build Vite + déploiement vers `gh-pages`).
(Tu peux réutiliser un template "Vite + GitHub Pages" depuis la doc Vite ou des exemples GitHub.)

En résumé : **build** = `npm run build`, **dossier à publier** = `build`, et les identifiants sont fixés au build via `VITE_APP_*`.

HTTPS (production)

- **Avec Docker** : voir `DEPLOIEMENT_SECURISE.md` (Certbot, volumes pour les certificats, config Nginx HTTPS dans le repo).
 - **Sans Docker** : utiliser Certbot (`certbot --nginx`) ou les certificats fournis par Netlify/Vercel.
-

Récap

Méthode	Commande / action principale	Où est le site
Docker (prod)	<code>docker compose -f docker-compose.prod.yml up --build -d</code>	http://localhost (port 80) ou ton domaine
Build + Nginx	<code>npm run build</code> puis copier <code>build/</code>	Répertoire configuré dans Nginx
Netlify / Vercel	Build = <code>npm run build</code> , dossier = <code>build</code>	URL fournie par la plateforme

Si tu dis sur quel type d'hébergement tu pars (VPS, Netlify, etc.), on peut détailler les commandes exactes pour ton cas.